

Step 3 — A Retriever Agent That Judges Its Own Search Results — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper LangGraph + RAG), **Advanced** (what makes "judges its own results" a real check instead of always saying yes). Read Basic first even if you already know RAG — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

The Retriever is a new node that sits between the graph's entry point and the agent loop from Step 2: given the task, it calls Doc08's `retrieve(query, k)` to get back a ranked list of chunks, then makes one more decision most plain RAG code skips — is what came back actually good enough to answer from? If yes, the graph continues with those chunks in state. If no, it routes to a separate "can't ground this" path instead of forcing a downstream node to answer with nothing real to work from.

This node only runs when it's actually needed — a conditional edge before it decides whether this task even calls for a document search at all, same idea as Step 2's `should_continue`, just deciding "search or skip" instead of "tool or done."

Need something to search over? If you don't have your own documents ready, load the Hugging Face dataset [rag-datasets/rag-mini-wikipedia](#) into your vector store — it's small, public domain, and made for exactly this kind of RAG practice. Your own text files work fine too; this is just a fast way to start testing.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The pieces to build:

```
# state.py (grown again)
class GraphState(TypedDict):
    task: str
    messages: Annotated[list, add_messages]
    found_chunks: list[Document]
    grounded: bool # did the Retriever judge its own results as good enough?

# agents/retriever_agent.py
def retrieve_node(state: GraphState) -> dict:
```

`retrieve_node` calls `retrieve(state["task"], k=3)` (Doc08's function, reused as `retriever.py`), then judges the result — either a real similarity-score threshold if your vector store returns scores, or a second, small model call asking "do these chunks actually contain material relevant to this question? yes/no." Return `{"found_chunks": chunks, "grounded": True}` or `{"found_chunks": chunks, "grounded": False}` accordingly — never just `{"found_chunks": chunks}` and let a downstream node guess.

Wire a routing function reading `state["grounded"]` (`"answer"` vs `"cant_ground"`) with `add_conditional_edges`, and a *separate* routing function before the Retriever entirely, deciding whether search is needed at all (Doc10's "search as a routed step" idea) — these are 2 different decisions, don't collapse them into one function.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

The single easiest way this step goes wrong invisibly: a relevance judgment that's really just "did `retrieve()` return a non-empty list?" That check passes even when the vector store's *closest* matches are still genuinely unrelated to the question — an empty list is the easy case to catch; a full list of confidently-wrong chunks is the actual failure mode "judges its own results" is supposed to catch. Write the relevance bar as something concrete you decided *before* looking at your first test's output: either a real numeric similarity-score cutoff (most vector stores can return scores alongside chunks — check what your `retrieve()` returns), or an explicit second-pass judgment prompt that names exactly what "good enough" means ("do these chunks actually mention the specific thing being asked, not just the same general topic?").

The second gap: once you have a real bar, you need a test case built to fail it on purpose — a question about something genuinely outside your knowledge base, not just a vague question about something that is in there. Testing only "does it find things when they're findable" never exercises the "can't ground this" path at all, and an untested path is exactly as reliable as an untested Verifier from Project 2 — meaning, not proven.

The extra pieces: - A relevance judgment that's either a numeric threshold on real scores, or a second small model call with an explicit yes/no rubric — never a bare "is the list non-empty" check. - A test question genuinely outside the knowledge base's topic, confirmed to route to `"cant_ground"`, alongside a normal in-scope question confirmed to route to `"answer"`.

Write your relevance rubric down in one sentence before writing any code for it, then check Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate wire the Retriever in as a real node with a real routing decision. Advanced is entirely about whether that decision is actually checking relevance, or accidentally just checking "did the search return anything at all" — the difference between a Retriever that judges its results and one that only looks like it does until the first genuinely out-of-scope question arrives.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
state adds: found_chunks (list), grounded (bool)

retrieve_node(state):
    chunks = retrieve(state["task"], k=3)
    good_enough = judge whether chunks actually answer the task
    return {"found_chunks": chunks, "grounded": good_enough}

route_after_retrieve(state):
    return "answer" if state["grounded"] else "cant_ground"

needs_search(state):
    return "search" if the task looks like it needs the knowledge base else "skip"

wire: entry -> (needs_search?) -> retrieve_node -> (route_after_retrieve?) -> answer
      \-> skip -> answer_placeholder directly
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
state.py:
class GraphState(TypedDict):
    task: str
    messages: Annotated[list, add_messages]
    found_chunks: list
    grounded: bool

agents/retriever_agent.py:
from retriever import retrieve

def retrieve_node(state):
    chunks = retrieve(state["task"], k=3)
    grounded = judge_relevance(state["task"], chunks)
    return {"found_chunks": chunks, "grounded": grounded}

def judge_relevance(task, chunks) -> bool:
    if not chunks:
        return False
    combined = "\n---\n".join(c.page_content for c in chunks)
    prompt = f"Question: {task}\n\nRetrieved text:\n{combined}\n\nDoes this t
    model = ChatOpenAI(model="gpt-4o-mini")
    answer = model.invoke(prompt).content.strip().lower()
    return answer.startswith("yes")

graph.py additions:
def needs_search(state):
    return "search" if looks_like_it_needs_the_kb(state["task"]) else "skip"

def route_after_retrieve(state):
    return "answer" if state["grounded"] else "cant_ground"

builder.add_node("retrieve", retrieve_node)
builder.add_conditional_edges(START, needs_search, {"search": "retrieve", "sk
builder.add_conditional_edges("retrieve", route_after_retrieve, {"answer": "a
```

Write `judge_relevance()`, `needs_search()`, and a placeholder

`answer_placeholder / cant_ground_node` (Step 4 replaces the former with the real Reasoner), then test both the "found something good" and "found something empty" paths, before moving to the Advanced version.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
judge_relevance rubric, decided up front:
    "good enough" means: at least one returned chunk specifically mentions
    the entity/fact the question is actually asking about – not just the
    same general topic area.

test_project3.py:
def test_in_scope_question_grounds():
    result = ...invoke with a question the KB genuinely answers...
    assert result["grounded"] is True

def test_out_of_scope_question_does_not_ground():
    result = ...invoke with a question about something NOT in the KB at all..
    assert result["grounded"] is False
```

Here's almost the whole thing — fill in the missing piece yourself:

```
def judge_relevance(task: str, chunks: list) -> bool:
    if not chunks:
        return False
    combined = "\n---\n".join(c.page_content for c in chunks)
    prompt = (
        f"Question: {task}\n\nRetrieved text:\n{combined}\n\n"
        "Does this text specifically mention the exact thing being asked about "
        "(not just the same general topic)? Reply only yes or no."
    )
    model = ChatOpenAI(model="gpt-4o-mini")
    answer = model.invoke(prompt).content.strip().lower()
    return answer.startswith("yes")

# your turn: write test_out_of_scope_question_does_not_ground(), using a question
# about a topic your knowledge base genuinely has nothing to do with
def test_out_of_scope_question_does_not_ground():
    ...
```

Fill in the out-of-scope test yourself, run both tests against your actual knowledge base, then compare against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate get a Retriever node with a real relevance check and real routing wired in. Advanced is about proving the relevance check is actually discriminating — written down as a concrete rubric before testing, and exercised against a genuinely out-of-scope question, not just eyeballed once against a question that was always going to succeed.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)